

Builder principle

design patterns are split into 3 categories

• Creational patterns

- Deal with creation/construction of objects
- Explicit (Constructor) v/s implicit (Dependency injection)
- whole rule (single statement) v/s piecewise (step by step)

• Structural patterns

- concerned with the structure
- many patterns are wrappers that mimic the underlying class interface
- Stand for the imp. of good API design.

• Behavioral patterns

- No set theme, they are just patterns suited for diff. types of behavior.

Builder is an API for doing step
wise step construction.

it is required when we have complicated
process of construction of object.

when creating an object that needs many
settings especially when most of them are
optional - the constructor becomes messy and
confusing.

example - Laptop

- Ram
 - disk space
 - graphic card
 - screen size
 - battery
 - keyboard
 - color
- } compulsory
} not compulsory,
} variable
} not compulsory

`l = Laptop(12, 512, None, 13", Yes, Black)`

- Extremely long parameter list
- hard to remember which params are
req. and which are optional
- no flexibility to set only req
params.

how to combine multiple builders?

class Person:

```
def __init__(self):  
    # address  
    self.street = None  
    self.postcode = None  
    self.city = None  
    # employment  
    self.company = None  
    self.desig = None  
    self.salary = None.
```

This property
defining a getter
method for job
description.

class PersonBuilder:

```
def __init__(self, person=Person()):  
    self._person = person.
```

```
def build(self):  
    return self.person
```

@property
def work(self):
 return
 PersonJobBuilder(
 self.person)

class PersonJobBuilder(PersonBuilder)

```
def __init__(self, person):  
    super().__init__(person)  
def at(self, company):  
    self.person.company = company  
    return self.  
def desig(self, designation):  
    self.person.desig = designation  
    return self.  
def sal(self, esal):  
    self.person.salary = salary
```

@property
def lives(self):
 return
 PersonAddBuilder(
 self.person)

return self

class Person AddrBuilder(Person Builder):

```
def __init__(self, person):  
    super().__init__(person)
```

```
def at(self, street):  
    self.person.street = street  
    return self
```

```
def withpost(self, postcode):  
    self.person.postcode = postcode  
    return self
```

```
def in-city(self, city):  
    self.person.city = city  
    return self
```

p = Person Builder()

person = p.

lives

• at('mumbai'). withpost('1234'). in-city('mumbai')

• works

• at('Company'). doing('engineer'). salary('123321')

• build()

One thing to notice here was that we were violating open-closed principle bcz, whenever we have to add a new builder, we have to modify the base class. "Person Builder".

alternative approach.

class Person:

```
def __init__(self):  
    self.name = None  
    self.position = None  
    self.dob = None
```

```
def __str__(self):  
    return f"({self.name} born on {self.dob} works as {self.position})"
```

@staticmethod

```
def new():  
    return PersonBuilder()
```

class PersonBuilder:

```
def __init__(self):  
    self.person = Person()
```

```
def build(self):  
    return self.person.
```

```
class PersonInfoBuilder(PersonBuilder):  
    def called(self, name):  
        self.person.name = name  
        return self
```

```
class PersonJobBuilder(PersonInfoBuilder):  
    def works_at(self, job):  
        self.person.position = job  
        return self
```

```
class PersonDOBBuilder(PersonJobBuilder):
```

```
    def born(self, dob):  
        self.person.dob = dob  
        return self
```

```
pb = PersonDOBBuilder()
```

```
me = pb.called("lt").works_at("company").born(1996)  
    .build()
```

```
print(me)
```